only safe rule to follow in a web browser is to *load source code only from sources you trust.*

You trust Agtron's data server to only return objects and not to override any prototypes or do anything else nasty to you. If you only poll it for new information periodically, as in listing 11.2, will you be getting data updates often enough? Surely there's some way to get updates in real time.

11.2.3. Going real time with WebSocket

If the data server could inform your dashboard application when there was new data, then you wouldn't need to poll. When the server informs the client in this way, the server is said to *push* data to the client. To achieve this, you need to open something called a WebSocket connection between the server and your dashboard.

To experiment with and learn about WebSocket, it's useful to have a local server that will push some data down a WebSocket connection so that you can play with it. You see a very basic Node.js-powered CoffeeScript program that pushes random data down a WebSocket connection in the following listing.

Listing 11.3. A random-number socket emitter

```
{EventEmitter} = require 'events'
WebSocketServer = (require 'websocket').server

seconds = (n) -> n*1000

emitRandomNumbers = (emitter, event, interval) ->
  setInterval ->
    emitter.emit event, Math.floor Math.random()*100
  , interval

source = new EventEmitter
emitRandomNumbers source, 'update', seconds(4)
attachSocketServer = (server) ->
  socketServer = new WebSocketServer httpServer: server
  socketServer.on 'request', (request) ->
    connection = request.accept 'graph', request.origin
    source.on 'update', (data) ->
      connection.sendUTF JSON.stringify data

exports.attachSocketServer = attachSocketServer
```

The websocket protocol handling here is provided by a library.

A function that will attach a socket server to an existing HTTP server.